Software Requirements Specification (SRS)
Utility Management System

Student Name: Farjana Islam Ria
Student ID: 22CSE022
Session: 2022-2023
Course Code: CSE356 | Course Title: Software Engineering Sessional
Department of Computer Science and Engineering, GSTU
Submitted To: Dr. Mrinal Kanti Baowaly (Professor, Dept. of CSE)

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define the functional and non-functional requirements, system architecture, user roles, interface specifications, and database models for the Utility Management System (UMS). This document serves as the official reference for software developers, system administrators, and utility operators.

1.2 Scope

The Utility Management System is a web-based portal designed to digitize and automate multi-utility management (Electricity, Water, and Gas). Key functional areas include:

- **Authentication:** Multi-role user login and role-based access control.
- **Meter Tracking:** Smart meter inventory, assignment, and consumption tracking.
- **Billing Engine:** Automated tariff billing engine based on consumption delta.
- **Notifications:** Instant invoice alerts and due-date reminders via SMTP.
- **Services:** Digital payments, complaint ticketing, GIS mapping, and analytics.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition & Context
SRS	Software Requirements Specification
UMS / utilis-app	Utility Management System Platform
UI / SPA	User Interface / Single Page Application
REST / API	Representational State Transfer / Application Programming Interface
GIS	Geographic Information System (Spatial Map Engine)
DFD / ERD	Data Flow Diagram / Entity Relationship Diagram

1.4 References

- IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications.
- IEEE Std 29148-2018: Systems and Software Engineering — Requirements Engineering.
- Express.js, MySQL 8.0, and Leaflet.js Technical Documentation.

1.5 Overview

This document is structured into 7 core sections: Section 2 covers overall system perspective; Section 3 detailing 10 functional modules; Section 4 specifies external interfaces; Section 5 defines non-functional metrics; Section 6 outlines regulatory policies; and Section 7 presents DFD and ERD Appendices.

2. Overall Description

2.1 Product Perspective

The Utility Management System is a decoupled full-stack web application. The backend REST service layer runs on Node.js/Express.js with MySQL (`utilis_db`), while the frontend presents a responsive SPA communicating via asynchronous JSON HTTP requests.

2.2 Product Functions

The system provides eleven major functional capabilities:

- Secure user authentication & role-based session state management.
- Customer directory linking profiles to assigned meters and service locations.
- Smart meter registration, serial tracking, and utility classification (Electricity/Water/Gas).
- Monthly meter reading logging and consumption delta computation (`currReading - prevReading`).
- Tiered tariff billing calculation, invoice creation, and overdue status enforcement.
- Automated email dispatch for bill notices, payment receipts, and urgent reminders.

- Multi-channel digital payment recording (bKash, Nagad, Bank, Card) with transaction IDs.
- Customer support ticketing with interactive JSON comment threads and status pipeline.
- Field maintenance dispatch for technician assignment and repair job tracking.
- GIS spatial outage map displaying customer meters, transformers, and fault zones.
- Financial analytics dashboard with Chart.js consumption graphs and PDF reports.

2.3 User Classes and Characteristics

User Role	Responsibilities & System Permissions
System Administrator (admin)	Full administrative control over customer records, meter setup, billing execution, financial reports, GIS map overview, technician assignment, and system settings.
Utility Field Staff (employee)	Logs monthly meter readings, updates technical maintenance ticket statuses, inspects field complaints, and views GIS meter locations.
Utility Customer (customer)	Views current & past bills, submits meter readings, pays utility invoices online, tracks payment receipts, logs complaints, and manages profile.

2.4 Operating Environment

- **Server:** Server runtime on Node.js v18+, Express.js framework, Nodemailer.
- **Database:** MySQL Server v8.0+ relational schema (`utilis_db`).
- **Client:** Web browsers (Chrome v100+, Firefox, Edge, Safari) with SPA execution.

2.5 Design and Implementation Constraints

- Dual-port architecture: Express API backend on port 5000 and HTTP frontend server on port 8080.
- Strict relational integrity with InnoDB foreign key constraints across 9 database tables.
- Non-negative consumption validation (`currReading >= prevReading`).

2.6 User Documentation

- Comprehensive setup instructions, API documentation, and user guides provided in `README.md`.

2.7 Assumptions and Dependencies

- Users access system using standard web browsers over HTTP/HTTPS connections.
- MySQL database server and SMTP email gateway services remain accessible.

3. System Features (Functional Requirements)

3.1 Module 1: User Authentication & Profile Management

- FR-01: Allow user login using valid email/username and password.
- FR-02: Maintain secure session storage (`sessionStorage`) with role permissions (`admin`, `employee`, `customer`).
- FR-03: Allow customer self-registration with name, email, phone, address, and meter serial.
- FR-04: Support profile view and edit capability for authenticated users.
- FR-05: Provide session logout clearing stored authentication tokens.

3.2 Module 2: Customer Account Management

- FR-06: Maintain customer directory linking profile records to authentication user IDs.
- FR-07: Allow administrators to search, edit, and manage customer accounts.
- FR-08: Support customer account activation and deactivation states.
- FR-09: Allow linking and re-assigning smart meters to customer profile accounts.

3.3 Module 3: Smart Meter & Reading Tracking

- FR-10: Maintain meter inventory recording serial number, utility type (Electricity, Water, Gas), and status.
- FR-11: Allow administrators and field staff to register new meters.
- FR-12: Record monthly meter readings capturing billing month and consumption units.
- FR-13: Display real-time meter operational flags (`active`, `faulty`, `inactive`).

3.4 Module 4: Consumption Billing & Tariff Calculation

- FR-14: Filter meter records by utility category (Electricity, Water, Gas) and assignment status.
- FR-15: Automatically compute consumption delta: `consumption = currReading - prevReading`.
- FR-16: Compute bill totals multiplying consumption delta by utility tariff rate.
- FR-17: Enforce due dates, billing month, and billing year on generated invoice records.
- FR-18: Allow administrators to trigger manual single or batch bill generation.
- FR-19: Dynamically set bill status flags (`unpaid`, `paid`, `overdue`).

3.5 Module 5: Automated Email Notification & Reminder System

- FR-20: Automatically dispatch invoice email notifications upon bill generation.
- FR-21: Provide admin endpoint (`/api/send-bill-reminder`) to send urgent email reminders for unpaid bills.
- FR-22: Render HTML email templates containing invoice ID, utility type, due date, and amount due.
- FR-23: Store in-app user notifications in database with read/unread tracking.

3.6 Module 6: Payment Processing & Financial Accounting

- FR-24: Support digital payment channel selection (bKash, Nagad, Rocket, Bank Transfer, Card).
- FR-25: Validate payment input data and record unique transaction reference IDs (`transactionId`).
- FR-26: Update bill status automatically from `unpaid` to `paid` upon successful verification.
- FR-27: Generate printable digital payment receipts for completed utility payments.

3.7 Module 7: Complaint Management & Support Ticketing

- FR-28: Allow customers to file complaints with subject, description, and utility category.
- FR-29: Track ticket pipeline states (`Pending`, `In Progress`, `Resolved`).
- FR-30: Allow admins to assign complaints to utility field staff.
- FR-31: Support interactive JSON comment threads attached to complaint tickets.

3.8 Module 8: Field Maintenance & Technical Operations

- FR-32: Allow logging maintenance job requests (Meter Inspection, Leak Repair, Line Maintenance).
- FR-33: Record maintenance type, description, customer link, technician ID, and timestamp.
- FR-34: Allow field staff to update maintenance task status and progress notes.
- FR-35: Display active technical maintenance metrics on admin overview cards.

3.9 Module 9: Geographic Information System (GIS) & Spatial Mapping

- FR-36: Integrate Leaflet GIS map displaying geographical distribution of customer meters.
- FR-37: Plot interactive map markers for meter health status and utility classifications.
- FR-38: Highlight active utility outage zones and transformer substation boundaries.

3.10 Module 10: Reports & Executive Analytics Engine

- FR-39: Render Chart.js analytics charts showing monthly consumption trends and revenue totals.
- FR-40: Generate executive KPIs for customer count, active meters, billed revenue, and unpaid balances.

- FR-41: Support exporting customer ledgers and billing summaries to PDF/printable format.

4. External Interface Requirements

4.1 User Interfaces

The web UI is built as a responsive Single Page Application (SPA) using HTML5, CSS3, and JavaScript ES6:

- Role-specific sidebar navigation rendering tailored workspace menus.
- KPI summary metric cards displaying real-time customer, meter, and revenue figures.
- Interactive data tables with instant filtering, search, and action buttons.
- Styled modal overlays for customer addition, meter log, bill pay, and ticket submission.
- Embedded Leaflet GIS map canvas with tooltips, zoom controls, and layer overlays.

4.2 Hardware Interfaces

- **Server Hardware:** Minimum 2-Core CPU, 4 GB RAM, 20 GB SSD storage.
- **Client Hardware:** PC, laptop, tablet, or smartphone (1024x768 display resolution standard).

4.3 Software Interfaces

- Windows 10/11, Linux (Ubuntu 20.04+), or macOS.
- Node.js runtime v18.0+, Express.js HTTP web framework.
- MySQL 8.0+ Server engine `InnoDB` with UTF-8 character encoding.

4.4 Communication Interfaces

- HTTP/1.1 or HTTPS protocol for REST API delivery.
- JSON data exchange (`Content-Type: application/json`) on backend port 5000.
- CORS enabled for frontend web server port 8080.
- SMTP (Simple Mail Transfer Protocol) for automated Nodemailer email dispatches.

5. Non-Functional Requirements

ID	Category	Specification & Quality Benchmark
NFR-01	Performance	API response times under 1.5 seconds for DB queries and under 3.0s for email dispatches.
NFR-02	Security	Passwords hashed securely; role-validated REST route protection and bounded sessions.
NFR-03	Reliability	InnoDB relational constraints ensure 99.9% data integrity and cascading updates.
NFR-04	Usability	Responsive layout with clear status badges, toast feedback, and modal dialogs.
NFR-05	Portability	Cross-browser compatibility across Google Chrome, Mozilla Firefox, Edge, and Safari.

6. Other Requirements

6.1 Regulatory Compliance & Tariff Policies

The system enforces standardized tariff structures approved by national utility commissions, applying per-unit charges based on utility classification (Electricity per kWh, Water per Litre, Gas per Unit).

6.2 Data Privacy & Billing Audit Trail

All financial transactions, bill generations, and payments record exact creation timestamps and user links to maintain an immutable accounting audit trail.

7. Appendices

7.1 Appendix A: Data Flow Diagram (DFD Level 1)

- Customer → [Login / Register] → User Authentication → Stores Session → `users` & `customers` DB.
- Field Staff → [Meter Reading Data] → Reading Log Process → Stores Consumption Delta → `meter_readings` DB.
- Billing Engine → [Fetch Readings & Tariff] → Billing Process → Computes Amount → `bills` DB → Sends Email.
- Customer → [Payment Details] → Payment Gateway → Updates Status → `payments` & `bills` DB.
- Customer → [Complaint / Request] → Support Ticket Process → Assigns Staff → `complaints` & `maintenance` DB.
- Administrator → [Request GIS & Analytics] → Reporting Engine → Aggregates Map & Financial Graphs.

7.2 Appendix B: Entity Relationship Diagram (ERD) & Database Schema

The `utilis_db` relational database comprises nine core tables structured as follows:

Database Table	Primary & Foreign Keys / Attributes	Functional Purpose
users	id (PK), name, username (UQ), email, password, role, status	Stores user authentication details and role permissions (admin, employee, customer).
customers	customer_id (PK), user_id (FK -> users.id), phone, address	Stores customer profile details linked to authentication user accounts.
meters	meter_id (PK), customer_id (FK), meter_number (UQ), utilityType, status	Stores utility meter inventory (Electricity, Water, Gas) and customer ownership.
meter_readings	reading_id (PK), meter_id (FK), month, unit_consumed	Tracks monthly consumption units logged per registered utility meter.
bills	bill_id (PK), customer_id (FK), amount, due_date, status, meterId (FK), prevReading, currReading, tariffRate	Stores calculated utility bills, reading deltas, tariff rates, due dates, and statuses.
payments	payment_id (PK), bill_id (FK), amount, payment_date, method, transactionId	Records payment transactions, channels (bKash, Nagad, Bank), and transaction IDs.
complaints	complaint_id (PK), customer_id (FK), subject, description, status, assignedTo, comments (JSON)	Manages customer support tickets, status pipeline, assigned staff, and comment logs.
notifications	id (PK), userId (FK -> users.id), message, type, date, isRead	Stores in-app system notifications and unread/read state indicators.
maintenance	maintenance_id (PK), customer_id (FK), request_type, description, status, assigned_to	Manages technical field maintenance requests, assigned technicians, and status tracking.